

Lista de Exercícios – Listas Simplesmente Encadeadas em Go

Estrutura de Dados 1 — Quarto Período

Esta lista contém 8 exercícios sobre as operações de lista simplesmente encadeada vistas na Aula 05, usando a struct:

```
type no struct {  
    valor int  
    proximo *no  
}  
  
type lista struct {  
    inicio *no  
}
```

O Exercício 1 deve ser resolvido em papel, à mão, sem uso de computador. Os Exercícios 2 a 8 devem ser implementados na linguagem Go e entregues via GitHub, seguindo o fluxo de trabalho descrito na seção 2.

1. Exercício em Papel

Exercício 1 — Escolha e Algoritmo (sem computador)

Escolha UMA das operações abaixo e escreva, à mão, o algoritmo correspondente utilizando a sintaxe da linguagem Go — incluindo a struct do nó e da lista e a função da operação escolhida. Não é necessário compilar nem executar: o objetivo é raciocinar sobre a lógica de manipulação de ponteiros, incluindo a ordem correta das atribuições (novo.proximo antes de anterior.proximo, quando aplicável).

Operações disponíveis para escolha:

- a) adicionarInicio — inserção no início da lista
- b) adicionarFim — inserção no fim da lista
- c) adicionarPosicao — inserção em uma posição específica
- d) removerInicio — remoção do primeiro elemento
- e) removerFim — remoção do último elemento
- f) removerPosicao — remoção de um elemento em uma posição específica
- g) posicao — busca de um valor, retornando sua posição
- h) valorNaPosicao — busca do valor armazenado em uma posição

Justifique, em poucas linhas, se a operação escolhida é $O(1)$ ou $O(n)$, e por quê.

2. Fluxo de Entrega via Git/GitHub (Exercícios 2 a 8)

O fluxo abaixo vale para esta lista e para todas as próximas atividades práticas do semestre:

1. Repositório único para o semestre: crie, uma única vez, um repositório no GitHub que será usado em todas as atividades práticas da disciplina ao longo do semestre (ex.: estrutura-de-dados-1-seunome).
2. Clone o repositório vazio para a sua máquina (git clone).
3. Para cada atividade, crie uma nova branch a partir da main (ex.: git checkout -b 01-adicionar-inicio-e-fim).
4. Implemente a solução na branch da atividade e, ao concluir, abra um Pull Request dessa branch para a main no GitHub.
5. Sempre que uma nova atividade for proposta ao longo do semestre, repita o processo: nova branch para a atividade e, ao final, novo Pull Request para a main — a main deve acumular o histórico de todas as atividades entregues.

Entrega: enviar o link do repositório.

3. Exercícios Práticos (Go + Git)

Exercício 2 — Struct e Inserção no Início e no Fim

Defina as structs no e lista mostradas acima e implemente adicionarInicio(valor int) e adicionarFim(valor int) como métodos de *lista. Escreva um main() que insira ao menos 5 valores (misturando início e fim) e confira o resultado.

Branch sugerida: 01-adicionar-inicio-e-fim

Exercício 3 — Inserção em Posição Específica

Implemente adicionarPosicao(valor, posicao int) bool. Siga a regra de ouro vista em aula: primeiro novo.proximo = anterior.proximo, depois anterior.proximo = novo — nessa ordem, para nunca perder o restante da lista. Trate posição inválida (posicao < 0 ou maior que o tamanho da lista) retornando false.

Branch sugerida: 02-adicionar-posicao

Exercício 4 — Remoção no Início e no Fim

Implemente removerInicio() (int, bool) e removerFim() (int, bool), cada uma retornando o valor removido e um bool indicando sucesso. Trate o caso de lista vazia (retornando 0, false) e o caso de lista com um único elemento.

Branch sugerida: 03-remover-inicio-e-fim

Exercício 5 — Remoção em Posição Específica

Implemente removerPosicao(posicao int) (int, bool), pulando o nó removido com anterior.proximo = anterior.proximo.proximo. Trate posição inválida e lista vazia.

Branch sugerida: 04-remover-posicao

Exercício 6 — Busca por Valor

Implemente `posicao(valorProcurado int) (int, bool)`, que percorre a lista comparando `atual.valor` a cada passo e retorna o índice onde o valor foi encontrado (ou 0, false se não existir).

Branch sugerida: 05-encontrar-posicao

Exercício 7 — Busca por Posição

Implemente `valorNaPosicao(posicaoProcurada int) (int, bool)`, que percorre a lista até o índice pedido e retorna o valor armazenado ali (ou 0, false se a posição não existir).

Branch sugerida: 06-valor-na-posicao

Exercício 8 — Tamanho, Impressão e Programa Integrador

Implemente `tamanho() int`, que percorre a lista e conta os nós, e `imprimir()`, que percorre a lista exibindo os valores no formato:

```
10 -> 20 -> 30 -> nil
```

Em seguida, una todas as operações dos Exercícios 2 a 7 em um único programa Go com um menu interativo no terminal (usando `fmt.Scan`). Teste cada operação com lista vazia, lista de 1 elemento e índices fora do intervalo — confirme que ela devolve false/0 como esperado nesses casos.

Branch sugerida: 07-tamanho-imprimir-integrador